WEEK 6 – REVIEW AND RATING MANAGEMENT SYSTEM

CITYCARE HOSPITAL MANAGEMENT SYSTEM

1. INTRODUCTION

The Review and Rating Management System is an important module of the CityCare Hospital Management System. It is designed to collect and manage patient feedback about medicines.

Patients can provide ratings and reviews for medicines they have purchased. The system stores patient opinions, analyses medicine ratings, and identifies highly rated medicines.

This module uses SQL operations, constraints, joins, and aggregate functions to manage medicine reviews and generate useful reports.

2. OBJECTIVES

The main objectives of this module are:

- To create a review and rating table.
- To store patient feedback and medicine ratings.
- To maintain ratings from 1 to 5 stars.
- To update and delete reviews when required.
- To retrieve reviews for specific medicines.
- To display patient names with their feedback.
- To calculate average medicine ratings.
- To identify highly rated medicines.
- To generate medicine rating analysis reports.
- To analyse the distribution of patient ratings.

3. TABLES USED

Table Name	Description
Patient	Stores patient information
Medicine	Stores medicine details
Medicine_Review	Stores patient reviews and ratings

The `Medicine_Review` table is connected to the existing Patient and Medicine tables using foreign keys.

4. CREATE REVIEW AND RATING TABLE

4.1 Create Medicine Review Table

SQL Query:

```
CREATE TABLE Medicine_Review (
    Review_ID NUMBER PRIMARY KEY,
    Patient_ID NUMBER NOT NULL,
    Medicine_ID NUMBER NOT NULL,
    Rating NUMBER(1),
    Review_Text VARCHAR2(250),
    Review_Date DATE DEFAULT SYSDATE,
    CONSTRAINT fk_review_patient
        FOREIGN KEY (Patient_ID)
        REFERENCES Patient(Patient_ID),
    CONSTRAINT fk_review_medicine
        FOREIGN KEY (Medicine_ID)
        REFERENCES Medicine(Medicine_ID),
    CONSTRAINT chk_medicine_rating
        CHECK (Rating BETWEEN 1 AND 5)
);
```

Explanation:

This query creates the Medicine_Review table to store review details, patient IDs, medicine IDs, ratings, review comments, and review dates.

The primary key uniquely identifies each review. Foreign keys establish relationships with the Patient and Medicine tables. The CHECK constraint restricts ratings to values between 1 and 5.

Output:

```
SQL> CREATE TABLE Medicine_Review (
  2      Review_ID NUMBER PRIMARY KEY,
  3      Patient_ID NUMBER NOT NULL,
  4      Medicine_ID NUMBER NOT NULL,
  5      Rating NUMBER(1),
  6      Review_Text VARCHAR2(250),
  7      Review_Date DATE DEFAULT SYSDATE,
  8      CONSTRAINT fk_review_patient
  9          FOREIGN KEY (Patient_ID)
 10          REFERENCES Patient(Patient_ID),
 11      CONSTRAINT fk_review_medicine
 12          FOREIGN KEY (Medicine_ID)
 13          REFERENCES Medicine(Medicine_ID),
 14      CONSTRAINT chk_medicine_rating
 15          CHECK (Rating BETWEEN 1 AND 5)
 16  );

Table created.

SQL> DESC Medicine_Review;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------
 REVIEW_ID                                 NOT NULL NUMBER
 PATIENT_ID                                NOT NULL NUMBER
 MEDICINE_ID                               NOT NULL NUMBER
 RATING                                             NUMBER(1)
 REVIEW_TEXT                                        VARCHAR2(250)
 REVIEW_DATE                                        DATE
```

5.1 Insert Medicine Reviews

SQL Query:

```
INSERT INTO Medicine_Review
VALUES (801, 301, 201, 5,
'Excellent medicine quality', SYSDATE);

INSERT INTO Medicine_Review
VALUES (802, 302, 202, 4,
'Good medicine and packaging', SYSDATE);

INSERT INTO Medicine_Review
VALUES (803, 303, 203, 5,
'Very satisfied with the medicine', SYSDATE);

INSERT INTO Medicine_Review
VALUES (804, 304, 204, 3,
'Packaging needs improvement', SYSDATE);

INSERT INTO Medicine_Review
VALUES (805, 305, 205, 4,
'Good overall experience', SYSDATE);
```

```
COMMIT;
```

Explanation:

The INSERT statements store sample patient feedback and medicine ratings in the Medicine_Review table.

Each review contains a unique review ID, patient ID, medicine ID, rating, review comment, and date.

Output:

```
SQL> INSERT INTO Medicine_Review
  2  VALUES (801, 301, 201, 5,
  3  'Excellent medicine quality', SYSDATE);

1 row created.

SQL>
SQL> INSERT INTO Medicine_Review
  2  VALUES (802, 302, 202, 4,
  3  'Good medicine and packaging', SYSDATE);

1 row created.

SQL>
SQL> INSERT INTO Medicine_Review
  2  VALUES (803, 303, 203, 5,
  3  'Very satisfied with the medicine', SYSDATE);

1 row created.

SQL>
SQL> INSERT INTO Medicine_Review
  2  VALUES (804, 304, 204, 3,
  3  'Packaging needs improvement', SYSDATE);

1 row created.

SQL>
SQL> INSERT INTO Medicine_Review
  2  VALUES (805, 305, 205, 4,
  3  'Good overall experience', SYSDATE);

1 row created.

SQL>
SQL> COMMIT;
```

5.2 Update Review Comments

SQL Query:

```
UPDATE Medicine_Review
SET Review_Text =
'Good quality but packaging can improve'
WHERE Review_ID = 804;

COMMIT;
```

Explanation:

The UPDATE statement modifies the review comment for the selected review without changing the remaining records.

Output:

```
SQL> UPDATE Medicine_Review
  2  SET Review_Text = 'Good quality but packaging can improve'
  3  WHERE Review_ID = 804;

1 row updated.

SQL>
SQL> COMMIT;

Commit complete.

SQL> SELECT * FROM Medicine_Review
  2  WHERE Review_ID = 804;

 REVIEW_ID PATIENT_ID MEDICINE_ID     RATING
---------- ---------- ----------- ----------
REVIEW_TEXT
--------------------------------------------------------------------------------
REVIEW_DA
---------
       804        304         204          3
Good quality but packaging can improve
26-OCT-26
```

5.3 Delete Invalid Reviews

SQL Query:

```
INSERT INTO Medicine_Review
VALUES (806, 301, 202, 2,
'Temporary invalid review', SYSDATE);

DELETE FROM Medicine_Review
WHERE Review_ID = 806;

COMMIT;
```

Explanation:

The DELETE statement removes a selected review from the database. This operation demonstrates how invalid or inappropriate reviews can be removed.

Output:

```
SQL> SELECT * FROM Medicine_Review WHERE Review_ID = 806;

no rows selected
```

6. RETRIEVE MEDICINE REVIEW DETAILS

6.1 Display All Reviews for a Medicine

SQL Query:

```
SELECT *
FROM Medicine_Review
WHERE Medicine_ID = 201;
```

Explanation:

This query retrieves all reviews associated with the selected medicine.

Output:

```
SQL> SELECT *
  2  FROM Medicine_Review
  3  WHERE Medicine_ID = 201;

 REVIEW_ID PATIENT_ID MEDICINE_ID     RATING
---------- ---------- ---------- ----------
REVIEW_TEXT
--------------------------------------------------------------------------------
REVIEW_DA
---------
       801        301         201          5
Excellent medicine quality
26-OCT-26
```

6.2 Display Patient Names With Their Reviews

SQL Query:

```
SELECT
    p.Patient_Name,
    m.Medicine_Name,
    r.Rating,
    r.Review_Text
FROM Medicine_Review r
JOIN Patient p
ON r.Patient_ID = p.Patient_ID
JOIN Medicine m
ON r.Medicine_ID = m.Medicine_ID;
```

Explanation:

This query uses INNER JOIN to combine patient, medicine, and review information. It displays patient names, medicine names, ratings, and review comments.

Output:

```
SQL> SELECT
  2      p.Patient_Name,
  3      m.Medicine_Name,
  4      r.Rating,
  5      r.Review_Text
  6  FROM Medicine_Review r
  7  JOIN Patient p
  8  ON r.Patient_ID = p.Patient_ID
  9  JOIN Medicine m
 10  ON r.Medicine_ID = m.Medicine_ID;

PATIENT_NAME
--------------------------------------------------------------------------------
MEDICINE_NAME
--------------------------------------------------------------------------------
    RATING
----------
REVIEW_TEXT
--------------------------------------------------------------------------------
Ananya Kumar
Paracetamol
         5
Excellent medicine quality


PATIENT_NAME
--------------------------------------------------------------------------------
MEDICINE_NAME
--------------------------------------------------------------------------------
    RATING
----------
REVIEW_TEXT
--------------------------------------------------------------------------------
Rahul Sharma
Amoxicillin
         4
Good medicine and packaging


PATIENT_NAME
```

```
MEDICINE_NAME
--------------------------------------------------------------------------------
    RATING
----------
REVIEW_TEXT
--------------------------------------------------------------------------------
Meena Devi
Cetirizine
         5
Very satisfied with the medicine


PATIENT_NAME
--------------------------------------------------------------------------------
MEDICINE_NAME
--------------------------------------------------------------------------------
    RATING
----------
REVIEW_TEXT
--------------------------------------------------------------------------------
Arun Raj
Omeprazole
         3
Good quality but packaging can improve


PATIENT_NAME
--------------------------------------------------------------------------------
MEDICINE_NAME
--------------------------------------------------------------------------------
    RATING
----------
REVIEW_TEXT
--------------------------------------------------------------------------------
Kavitha S
Azithromycin
         4
Good overall experience
```

6.3 Find Medicines Having Maximum Reviews

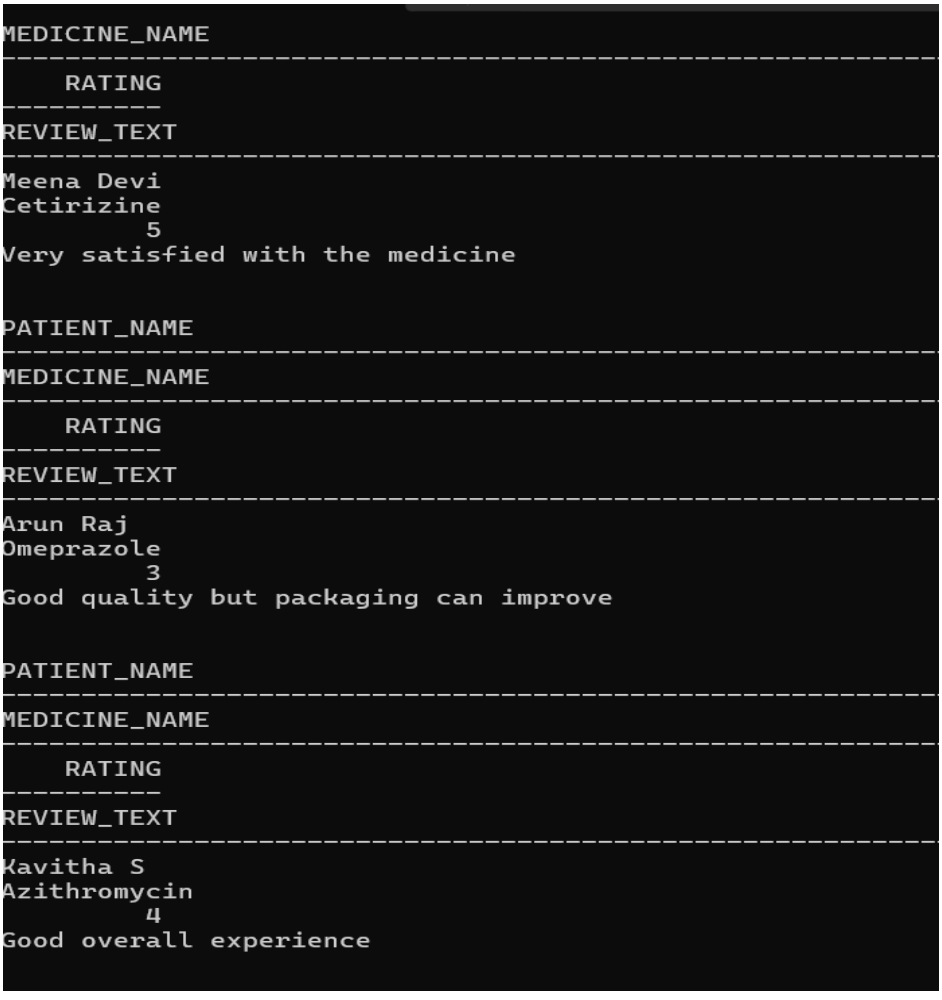
SQL Query:

```
SELECT
    m.Medicine_Name,
    COUNT(r.Review_ID) AS Review_Count
FROM Medicine m
JOIN Medicine_Review r
ON m.Medicine_ID = r.Medicine_ID
GROUP BY m.Medicine_Name
ORDER BY Review_Count DESC;
```

Explanation:

This query counts the reviews received by each medicine and arranges the results in descending order.

Output:



6.4 Display Recent Patient Feedback

SQL Query:

```
SELECT
    p.Patient_Name,
    m.Medicine_Name,
    r.Rating,
    r.Review_Text,
    r.Review_Date
FROM Medicine_Review r
JOIN Patient p
ON r.Patient_ID = p.Patient_ID
JOIN Medicine m
ON r.Medicine_ID = m.Medicine_ID
ORDER BY r.Review_Date DESC
FETCH FIRST 5 ROWS ONLY;
```

Explanation:

This query displays the five most recent patient feedback records based on the review date.

Output:

```
SQL> SELECT *
  2  FROM (
  3      SELECT
  4          p.Patient_Name,
  5          m.Medicine_Name,
  6          r.Rating,
  7          r.Review_Text,
  8          r.Review_Date
  9      FROM Medicine_Review r
 10      JOIN Patient p
 11          ON r.Patient_ID = p.Patient_ID
 12      JOIN Medicine m
 13          ON r.Medicine_ID = m.Medicine_ID
 14      ORDER BY r.Review_Date DESC, r.Review_ID DESC
 15  )
 16  WHERE ROWNUM <= 5;

PATIENT_NAME
--------------------------------------------------------------------------------
MEDICINE_NAME
--------------------------------------------------------------------------------
    RATING
----------
REVIEW_TEXT
--------------------------------------------------------------------------------
REVIEW_DA
---------
Kavitha S
Azithromycin
         4

PATIENT_NAME
--------------------------------------------------------------------------------
MEDICINE_NAME
--------------------------------------------------------------------------------
    RATING
----------
REVIEW_TEXT
--------------------------------------------------------------------------------
```

```
--------------------------------------------------------------------------------
REVIEW_DA
---------
Good overall experience
26-OCT-26


PATIENT_NAME
--------------------------------------------------------------------------------
MEDICINE_NAME
--------------------------------------------------------------------------------
    RATING
----------
REVIEW_TEXT
--------------------------------------------------------------------------------
REVIEW_DA
---------
Arun Raj
Omeprazole
         3

PATIENT_NAME
--------------------------------------------------------------------------------
MEDICINE_NAME
--------------------------------------------------------------------------------
    RATING
----------
REVIEW_TEXT
--------------------------------------------------------------------------------
REVIEW_DA
---------
Good quality but packaging can improve
26-OCT-26


PATIENT_NAME
--------------------------------------------------------------------------------
MEDICINE_NAME
--------------------------------------------------------------------------------
    RATING
----------
```

```
--------------------------------------------------------------------------------
REVIEW_DA
---------
Meena Devi
Cetirizine
         5

PATIENT_NAME
--------------------------------------------------------------------------------
MEDICINE_NAME
--------------------------------------------------------------------------------
    RATING
----------
REVIEW_TEXT
--------------------------------------------------------------------------------
REVIEW_DA
---------
Very satisfied with the medicine
26-OCT-26


PATIENT_NAME
--------------------------------------------------------------------------------
MEDICINE_NAME
--------------------------------------------------------------------------------
    RATING
----------
REVIEW_TEXT
--------------------------------------------------------------------------------
REVIEW_DA
---------
Rahul Sharma
Amoxicillin
         4

PATIENT_NAME
--------------------------------------------------------------------------------
MEDICINE_NAME
--------------------------------------------------------------------------------
    RATING
```

```
MEDICINE_NAME
--------------------------------------------------------------
    RATING
----------
REVIEW_TEXT
--------------------------------------------------------------
REVIEW_DA
---------
Good medicine and packaging
26-OCT-26


PATIENT_NAME
--------------------------------------------------------------
MEDICINE_NAME
--------------------------------------------------------------
    RATING
----------
REVIEW_TEXT
--------------------------------------------------------------
REVIEW_DA
---------
Ananya Kumar
Paracetamol
         5

PATIENT_NAME
--------------------------------------------------------------
MEDICINE_NAME
--------------------------------------------------------------
    RATING
----------
REVIEW_TEXT
--------------------------------------------------------------
REVIEW_DA
---------
```

6.5 Retrieve Reviews With Ratings Above 4

SQL Query:

```
SELECT *
FROM Medicine_Review
WHERE Rating > 4;
```

Explanation:

This query retrieves reviews with ratings greater than four stars.

Output:

```
SQL> SELECT *
  2  FROM Medicine_Review
  3  WHERE Rating > 4;

 REVIEW_ID PATIENT_ID MEDICINE_ID     RATING
---------- ---------- ----------- ----------
REVIEW_TEXT
--------------------------------------------------------------------------------
REVIEW_DA
---------
       801        301         201          5
Excellent medicine quality
26-OCT-26

       803        303         203          5
Very satisfied with the medicine
26-OCT-26

 REVIEW_ID PATIENT_ID MEDICINE_ID     RATING
---------- ---------- ----------- ----------
REVIEW_TEXT
--------------------------------------------------------------------------------
REVIEW_DA
---------
```

7. CALCULATE AVERAGE MEDICINE RATINGS

7.1 Calculate Average Rating for Each Medicine

SQL Query:

```
SELECT
    m.Medicine_Name,
    ROUND(AVG(r.Rating), 2) AS Average_Rating
FROM Medicine m
JOIN Medicine_Review r
ON m.Medicine_ID = r.Medicine_ID
GROUP BY m.Medicine_Name;
```

Explanation:

The AVG function calculates the average rating for each medicine. The ROUND function displays the result to two decimal places.

Output:

```
SQL> SELECT
  2       m.Medicine_Name,
  3       ROUND(AVG(r.Rating), 2) AS Average_Rating
  4  FROM Medicine m
  5  JOIN Medicine_Review r
  6  ON m.Medicine_ID = r.Medicine_ID
  7  GROUP BY m.Medicine_Name;

MEDICINE_NAME
--------------------------------------------------------------
AVERAGE_RATING
--------------
Amoxicillin
             4

Azithromycin
             4

Paracetamol
             5


MEDICINE_NAME
--------------------------------------------------------------
AVERAGE_RATING
--------------
Omeprazole
             3

Cetirizine
             5
```

7.2 Count the Number of Reviews per Medicine

SQL Query:

```
SELECT
    m.Medicine_Name,
    COUNT(r.Review_ID) AS Total_Reviews
FROM Medicine m
LEFT JOIN Medicine_Review r
```

```
ON m.Medicine_ID = r.Medicine_ID
GROUP BY m.Medicine_Name;
```

Explanation:

The COUNT function calculates the total number of reviews received by each medicine. LEFT JOIN includes medicines that have not received any reviews.

Output:

```
SQL> SELECT
  2       m.Medicine_Name,
  3       COUNT(r.Review_ID) AS Total_Reviews
  4  FROM Medicine m
  5  LEFT JOIN Medicine_Review r
  6  ON m.Medicine_ID = r.Medicine_ID
  7  GROUP BY m.Medicine_Name;

MEDICINE_NAME
--------------------------------------------------------------------
TOTAL_REVIEWS
-------------
Amoxicillin
            1

Azithromycin
            1

Paracetamol
            1


MEDICINE_NAME
--------------------------------------------------------------------
TOTAL_REVIEWS
-------------
Omeprazole
            1

Cetirizine
            1

Ibuprofen
            0


6 rows selected.
```

7.3 Find Highest-Rated Medicines

SQL Query:

```
SELECT
    m.Medicine_Name,
    ROUND(AVG(r.Rating), 2) AS Average_Rating
FROM Medicine m
JOIN Medicine_Review r
```

```
ON m.Medicine_ID = r.Medicine_ID
GROUP BY m.Medicine_Name
HAVING AVG(r.Rating) = (
    SELECT MAX(AVG(Rating))
    FROM Medicine_Review
    GROUP BY Medicine_ID
);
```

Explanation:

This query identifies the medicines with the highest average rating using aggregate functions and a subquery.

Output:

```
SQL> SELECT
  2      m.Medicine_Name,
  3      ROUND(AVG(r.Rating), 2) AS Average_Rating
  4  FROM Medicine m
  5  JOIN Medicine_Review r
  6  ON m.Medicine_ID = r.Medicine_ID
  7  GROUP BY m.Medicine_Name
  8  HAVING AVG(r.Rating) = (
  9      SELECT MAX(AVG(Rating))
 10      FROM Medicine_Review
 11      GROUP BY Medicine_ID
 12  );

MEDICINE_NAME
--------------------------------------------------------------------
AVERAGE_RATING
--------------
Paracetamol
             5

Cetirizine
             5
```

7.4 Identify Medicines With Average Rating Above 4

SQL Query:

```
SELECT
    m.Medicine_Name,
    ROUND(AVG(r.Rating), 2) AS Average_Rating
FROM Medicine m
JOIN Medicine_Review r
ON m.Medicine_ID = r.Medicine_ID
GROUP BY m.Medicine_Name
HAVING AVG(r.Rating) > 4;
```

Explanation:

This query identifies medicines whose average patient rating is greater than four.

Output:

```
SQL> SELECT
  2      m.Medicine_Name,
  3      ROUND(AVG(r.Rating), 2) AS Average_Rating
  4  FROM Medicine m
  5  JOIN Medicine_Review r
  6  ON m.Medicine_ID = r.Medicine_ID
  7  GROUP BY m.Medicine_Name
  8  HAVING AVG(r.Rating) > 4;

MEDICINE_NAME
--------------------------------------------------------------------
AVERAGE_RATING
--------------
Paracetamol
             5

Cetirizine
             5
```

8. MEDICINE RATING ANALYSIS REPORT

8.1 Report 1 – Medicine Rating Analysis

SQL Query:

```
SELECT
    m.Medicine_Name,
    COUNT(r.Review_ID) AS Number_Of_Reviews,
    ROUND(AVG(r.Rating), 2) AS Average_Rating
FROM Medicine m
LEFT JOIN Medicine_Review r
ON m.Medicine_ID = r.Medicine_ID
GROUP BY m.Medicine_Name
ORDER BY Average_Rating DESC NULLS LAST;
```

Explanation:

This report displays medicine names, the number of reviews, and their average ratings. It helps compare patient feedback across medicines.

Output:

```
SQL> SELECT
  2      m.Medicine_Name,
  3      COUNT(r.Review_ID) AS Number_Of_Reviews,
  4      ROUND(AVG(r.Rating), 2) AS Average_Rating
  5  FROM Medicine m
  6  LEFT JOIN Medicine_Review r
  7  ON m.Medicine_ID = r.Medicine_ID
  8  GROUP BY m.Medicine_Name
  9  ORDER BY Average_Rating DESC NULLS LAST;

MEDICINE_NAME
--------------------------------------------------------------------
NUMBER_OF_REVIEWS AVERAGE_RATING
----------------- --------------
Paracetamol
                1              5

Cetirizine
                1              5

Azithromycin
                1              4


MEDICINE_NAME
--------------------------------------------------------------------
NUMBER_OF_REVIEWS AVERAGE_RATING
----------------- --------------
Amoxicillin
                1              4

Omeprazole
                1              3

Ibuprofen
                0


6 rows selected.
```

8.2 Report 2 – Medicines Requiring Improvement

SQL Query:

```
SELECT
    m.Medicine_Name,
    COUNT(r.Review_ID) AS Review_Count,
    ROUND(AVG(r.Rating), 2) AS Average_Rating
FROM Medicine m
JOIN Medicine_Review r
ON m.Medicine_ID = r.Medicine_ID
GROUP BY m.Medicine_Name
HAVING AVG(r.Rating) <= 3;
```

Explanation:

This report identifies medicines with average feedback ratings of three or below. These ratings can help identify areas of patient dissatisfaction for further review.

Output:

```
SQL> SELECT
  2      m.Medicine_Name,
  3      COUNT(r.Review_ID) AS Review_Count,
  4      ROUND(AVG(r.Rating), 2) AS Average_Rating
  5  FROM Medicine m
  6  JOIN Medicine_Review r
  7  ON m.Medicine_ID = r.Medicine_ID
  8  GROUP BY m.Medicine_Name
  9  HAVING AVG(r.Rating) <= 3;

MEDICINE_NAME
--------------------------------------------------------------------
REVIEW_COUNT AVERAGE_RATING
------------ --------------
Omeprazole
           1              3
```

9. RATING DISTRIBUTION REPORT

9.1 Complete Rating Distribution

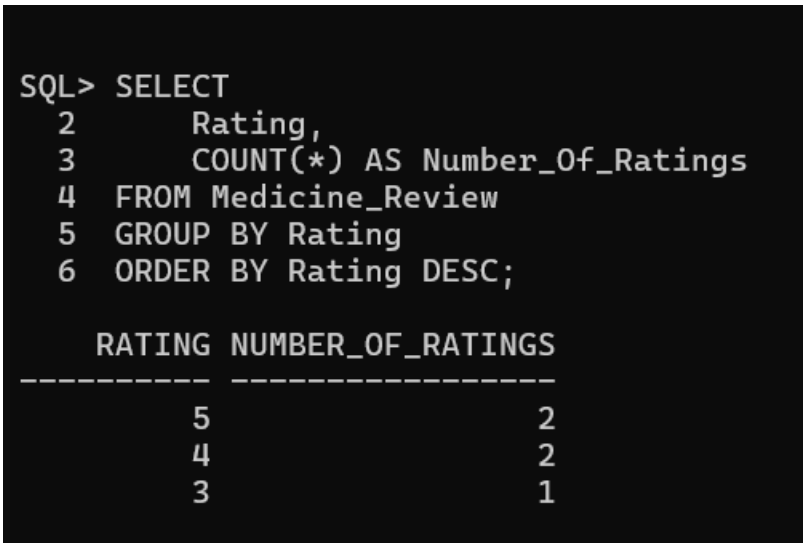
SQL Query:

```
SELECT
    Rating,
    COUNT(*) AS Number_Of_Ratings
FROM Medicine_Review
GROUP BY Rating
ORDER BY Rating DESC;
```

Explanation:

This query displays the number of reviews received for each rating value.

Output:



```
SQL> SELECT
  2      Rating,
  3      COUNT(*) AS Number_Of_Ratings
  4  FROM Medicine_Review
  5  GROUP BY Rating
  6  ORDER BY Rating DESC;

    RATING NUMBER_OF_RATINGS
---------- -----------------
         5                 2
         4                 2
         3                 1
```

9.2 Count Five-Star Ratings

SQL Query:

```
SELECT COUNT(*) AS Five_Star_Ratings
FROM Medicine_Review
WHERE Rating = 5;
```

Explanation:

This query calculates the total number of five-star reviews.

Output:

```
SQL> SELECT COUNT(*) AS Five_Star_Ratings
  2  FROM Medicine_Review
  3  WHERE Rating = 5;

FIVE_STAR_RATINGS
-----------------
                2
```

9.3 Count Four-Star Ratings

SQL Query:

```
SELECT COUNT(*) AS Four_Star_Ratings
FROM Medicine_Review
WHERE Rating = 4;
```

Explanation:

This query calculates the total number of four-star reviews.

Output:

```
SQL> SELECT COUNT(*) AS Low_Rated_Medicines
  2  FROM (
  3      SELECT Medicine_ID
  4      FROM Medicine_Review
  5      GROUP BY Medicine_ID
  6      HAVING AVG(Rating) < 3
  7  );

LOW_RATED_MEDICINES
-------------------
                  0
```

9.4 Count Low-Rated Medicines

SQL Query:

```
SELECT COUNT(*) AS Low_Rated_Medicines
FROM (
    SELECT Medicine_ID
    FROM Medicine_Review
    GROUP BY Medicine_ID
    HAVING AVG(Rating) < 3
);
```

Explanation:

This query counts medicines whose average patient rating is below three.

Output:


```
SQL> SELECT COUNT(*) AS Low_Rated_Medicines
  2  FROM (
  3      SELECT Medicine_ID
  4      FROM Medicine_Review
  5      GROUP BY Medicine_ID
  6      HAVING AVG(Rating) < 3
  7  );

LOW_RATED_MEDICINES
-------------------
                  0
```

10. RESULT

The Review and Rating Management System module demonstrates how SQL can be used to store, update, delete, and retrieve patient feedback.

The module includes rating validation using constraints, review retrieval using joins, average rating calculations using aggregate functions, and reports for analysing medicine feedback.

The actual results of these queries are shown in the corresponding SQL*Plus screenshots.

11. CONCLUSION

The Review and Rating Management System provides a structured approach to managing patient feedback in the CityCare Hospital Management System.

The implementation demonstrates the use of CREATE, INSERT, UPDATE, DELETE, SELECT, JOIN, GROUP BY, HAVING, COUNT, and AVG operations in Oracle SQL.

The module helps organise patient reviews, analyse medicine ratings, identify frequently reviewed medicines, and generate useful feedback reports.

Thus, Week 6 demonstrates the practical application of database operations and analytical SQL queries in a hospital management environment.